

One-page note — Kill the Quote Spreadsheet

What this is: A short note for anyone reviewing the live demo. **Live demo:** <https://kill-the-quote-spreadsheet-lac.vercel.app/> **Code:** <https://github.com/Anirudhmen94/Aerchainfinal>

What we are doing (in one paragraph)

Procurement buyers still spend days copying vendor quotes into Excel. They send an RFx to five suppliers and get back messy replies — Excel that ignores the template, a PDF with a discount buried in a footnote, a Word paragraph, a phone photo of a rate card, or a one-line email. Then a manager asks one “what if” question and another afternoon disappears.

This prototype ends that week. A buyer drafts the RFx with an AI co-pilot, vendors reply however they like, the system reads every format into one side-by-side comparison (same lines, same units, same currency), and the buyer asks questions in plain English through to an award decision they can defend.

Demo size: corrugated packaging for a Chakan snacks plant · **5 vendors** · **30 line items** · **8 quality questions**.

How the product works (walk the tabs)

Tab	What happens
Draft	Buyer describes the need in plain language → AI drafts ~30 lines + quality questionnaire + 5 vendors.
Outbox	Preview cover emails → “send” writes stub files (no real email server). Later: award notices and manager alerts land here too.
Inbox	Seed or upload the five ugly reply formats → Parse → structured quotes.
Compare	One INR matrix for all vendors. Click a cell to see the original source (snippet + file). Unclear values are marked uncertain/missing — not hidden.
Ask	Chat in normal English over the comparison (e.g. cheapest split among vendors who passed quality). Answers come from real extracted data, not canned scripts.
Award	Pick a vendor per line → save. If you want a failing/incomplete vendor on a line, the system notifies a manager via Outbox (there is no fake Approve button).
Audit	Log of saves, notices, and overrides — so decisions are reviewable.

Trust rules we built for: every price should link back to evidence; math (FX, unit conversion, cheapest-line) runs in code, not in the AI; vendors who fail knockout questions are clearly flagged and kept off the default award list.

What we deliberately left out (and why)

- **Real email (SMTP/IMAP).** Outbox saves stub files only. We prove the messages and workflow without building a mail product.
- **Freeze / Unfreeze on Award.** Removed on purpose. Saving the award + Audit is enough for a demo; a freeze ceremony added ceremony, not trust.
- **Vendor portal or forced reply template.** Vendors stay free to reply in any format; our job is to absorb the mess.
- **Approve / Reject UI for managers on Award.** Managers get a notify stub in Outbox only. The buyer owns the screen; we did not fake a full approval workflow.
- **ERP, payments, purchase orders.** Scope stops at a defensible award decision.
- **Multi-user login / permissions.** Single-buyer demo so the ugly-edge story stays center stage.
- **Hardcoded answers in Ask.** Free questions must run on the extracted data — canned answers would break the assignment.
- **Silently applying footnote discounts into award totals.** Discounts stay visible as caveats; quiet math into “official” totals would destroy trust.
- **One-click “run everything” on Home.** The reviewer walks the real path tab by tab.

Where the more interesting problem actually is

The more interesting problem is not the spreadsheet itself. The spreadsheet is where procurement teams manually consolidate a deeper problem: **supplier responses are not immediately decision-comparable**.

A price is only useful when the buyer knows what it covers, which unit it uses, whether freight and tooling are included, whether the specification matches, whether the supplier is qualified, and what evidence supports it. Without that context, an apparently precise comparison can create false confidence.

I would therefore define the product as an **evidence-backed procurement decision system**. Its core responsibility is to establish comparability before optimizing price. It should separate quoted values from normalized values, distinguish eligible vendors from merely cheap vendors, flag missing or ambiguous information, and preserve the evidence behind every recommendation.

The longer-term opportunity is an **exception-resolution loop**: extract the quote, detect what prevents comparison, generate a targeted clarification request, update the offer when the supplier responds, and preserve the decision history. In other words, the product should not only reduce spreadsheet work; it should reduce the risk of making a high-value award decision on incomplete or falsely comparable data.

My prototype focuses on quote comparison because that is the clearest entry point, but the broader product is **extract → validate → clarify → compare → recommend → audit**.